Assignment 4

1…………………….

```
MOV AX, 4000H
MOV DS, AX

MOV AX, 5000H
MOV ES, AX

MOV SI, 5000H
MOV DI, 6000H

MOV CX, 05H

CLD
REP MOVSB

HLT
```

2……………………..

```
MOV AX, 3000H
MOV DS, AX

MOV SI, 4000H
MOV BL, 24H
MOV CL, 00H

J2:
MOV AL, [SI]

CMP AL, BL
JZ J1

INC CL
INC SI

JMP J2

J1:
MOV [5000H], CL

HLT
```

3………….

```
MOV AX, 3000H
MOV DS, AX

MOV CX, 05H

MOV SI, 4004H
MOV DI, 5000H

L1:
MOV AL, [SI]
MOV [DI], AL

DEC SI
INC DI

DEC CX
JNZ L1

HLT
```

4……………………….

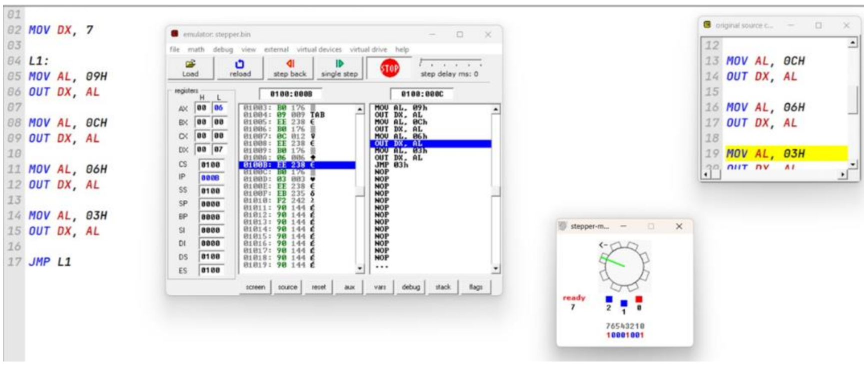
```
MOV AX, 5000H
MOV DS, AX

MOV SI, 8000H

MOV BL, [SI]

MOV AL, 01H
MOV AH, 00H

CMP BL, 00H
JZ END_FACT

L1:
MUL BL
DEC BL
JNZ L1

END_FACT:
MOV [8002H], AX

HLT
```

5……………….

```
MOV AX, 3000H
MOV DS, AX

MOV SI, 4000H

MOV AL, [SI]

MOV BL, 01H
MOV CL, 00H

L1:
CMP AL, BL
JB DONE

SUB AL, BL
INC CL

ADD BL, 02H
JMP L1

DONE:
MOV [4002H], CL

HLT
```

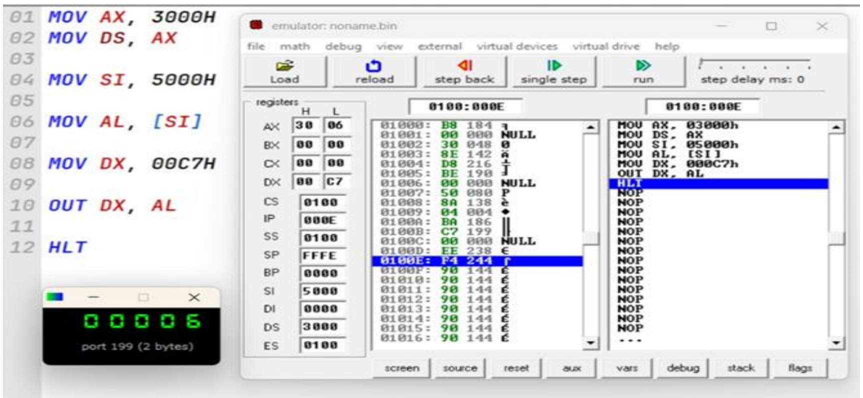
Assignment 5

Stepper motor……………..

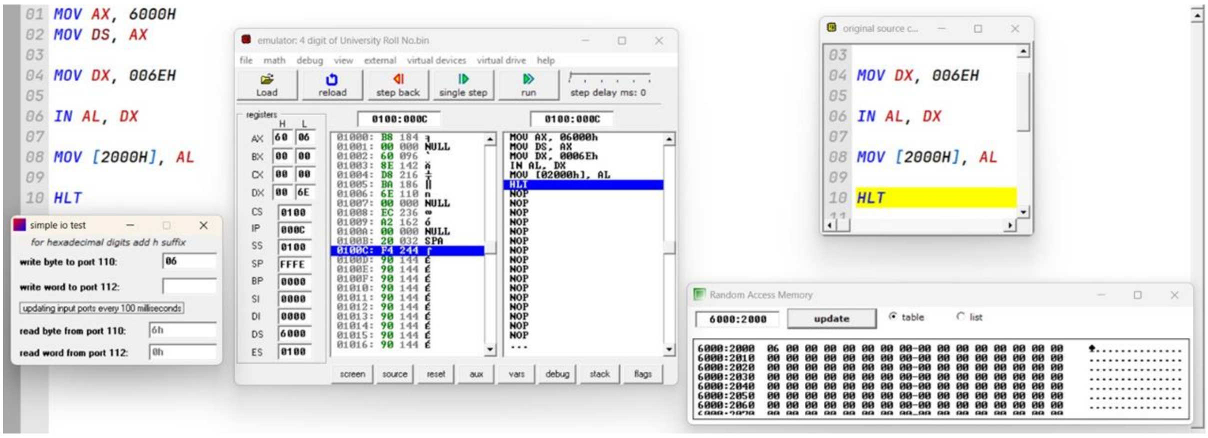


```
MOV DX, 7

L1:
MOV AL, 09H
OUT DX, AL

MOV AL, 0CH
OUT DX, AL

MOV AL, 06H
OUT DX, AL

MOV AL, 03H
OUT DX, AL

JMP L1
```

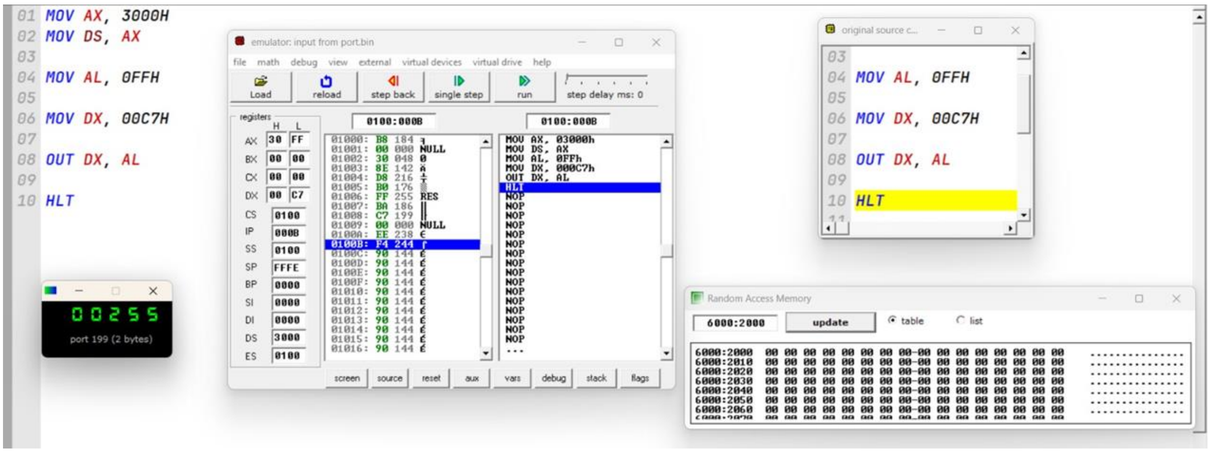
Led display…………..



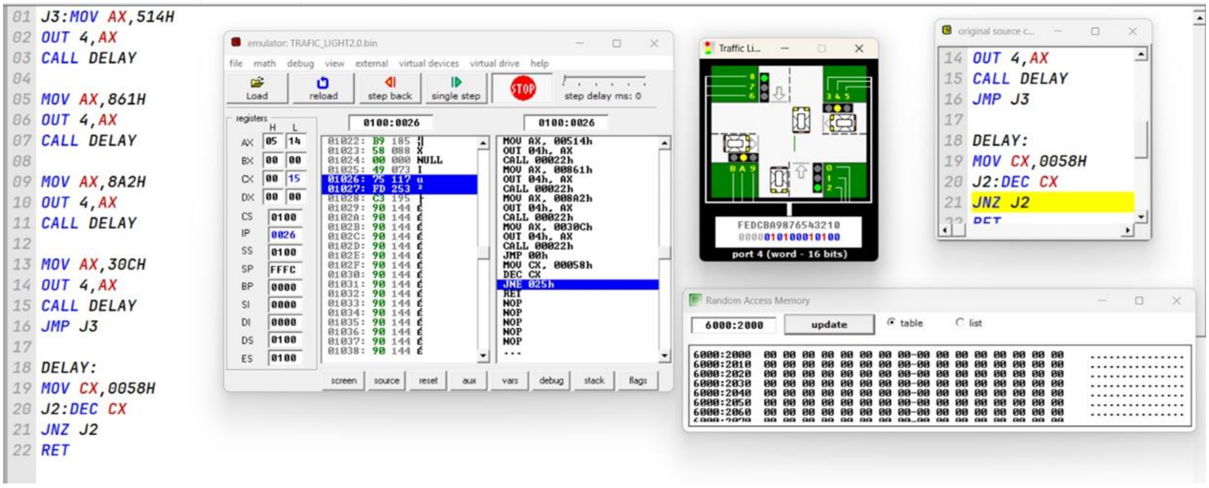
MOV AX, 3000H
MOV DS, AX
MOV SI, 5000H
MOV AL, [SI]
MOV DX, 00C7H
OUT DX, AL
HLT
00006
port 199 (2 bytes)



MOV AX, 6000H
MOV DS, AX
MOV DX, 006EH
IN AL, DX
MOV [2000H], AL
HLT
simple io test
for hexadecimal digits add h suffix
write byte to port 110:
write word to port 112:
read byte from port 110:
read word from port 112:
Random Access Memory



MOV AX, 3000H
MOV DS, AX
MOV AL, 0FFH
MOV DX, 00C7H
OUT DX, AL
HLT
00255
port 199 (2 bytes)
Random Access Memory

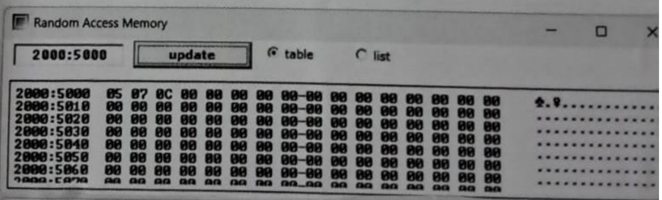

J3:MOV AX,514H
OUT 4,AX
CALL DELAY
MOV AX,861H
OUT 4,AX
CALL DELAY
MOV AX,8A2H
OUT 4,AX
CALL DELAY
MOV AX,30CH
OUT 4,AX
CALL DELAY
JMP J3
DELAY:
MOV CX,0058H
J2:DEC CX
JNZ J2
RET
emulator: TRAFIC_LIGHT2.0.bin
Load
reload
step back
single step
STOP
step delay ms: 0
registers
Traffic Li...
FEDCBA9876543210
port 4 (word - 16 bits)
original source c...
Random Access Memory
6000:2000
update
table
list

Assignment 1: Write a program to add two 8-bit numbers where segment address is 2000H and offset memory address is 5000H and store result into memory address.

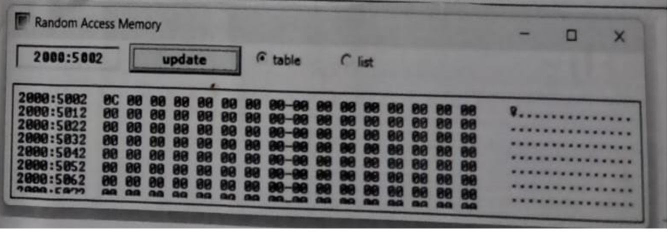
Program:

```
MOV AX, 2000H
MOV DS, AX
MOV SI, 5000H
MOV AL, [SI]
MOV BL, [SI+1]
ADD AL, BL
MOV [SI+2], AL
HLT
```

Input



Output



c

Assignment 2: Write a program to add two 16-bit numbers where segment address is 2000H and offset memory address is 5000H and store result into memory address.

Program:

```
MOV AX, 2000H
MOV DS, AX
MOV SI, 5000H
MOV AX, [SI]
MOV BX, [SI+2]
ADD AX, BX
MOV [SI+4], AX
HLT
```

Input

Random Access Memory

2000:5000 update ◉ table ○ list 25006

```
2000:5000  00 01 00 11 00 00 00 00-00 00 00 00 00 00 00 00
2000:5010  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5020  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Output

Random Access Memory

2000:5004 update ◉ table ○ list

```
2000:5004  00 12 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5014  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5024  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5034  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5044  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5054  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
2000:5064  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Assignment 3: Write a program to subtract two 8-bit numbers where segment address is 3000H and offset memory address is 9000H and store result into memory address.

Program:

```
MOV AX, 3000 H
MOV DS, AX
MOV SI, 9000 H
MOV AL, [SI]
MOV BL, [SI+1]
SUB AL, BL
MOV [SI+2], AL
HLT
```

Input

Random Access Memory

3000:9000 | update | table | list | 39003

3000:9000	08	05	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9010	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9020	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9030	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9040	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9050	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9060	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00

Output

Random Access Memory

3000:9002 | update | table | list

3000:9002	03	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9012	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9022	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9032	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9042	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9052	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00
3000:9062	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00

Assignment 4: Write a program to multiply two 8-bit numbers where segment address is 4000H and offset memory address is 3000H and store result into memory address.

Program:

```
MOV AX, 4000H
MOV DS, AX
MOV SI, 3000H
MOV AL, [SI]
MOV BL, [SI+1]
MUL BL
MOV [SI+2], AL
MOV [SI+3], AH
HLT
```

Input

Random Access Memory

4000:3000 | update | table | list | 43003

```
4000:3000  07 06 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3010  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3020  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Output

Random Access Memory

4000:3002 | update | table | list

```
4000:3002  2A 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3012  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3022  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3032  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3042  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3052  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
4000:3062  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Assignment 5: Write a program to Division two 16-bit numbers where segment address is 5000H and offset memory address is 3000H and store result into memory address.

Program:

```
MOV AX, 5000H
MOV DS, AX
MOV SI, 3000H
MOV AX, [SI]
MOV DX, 0000H
MOV BX, [SI+2]
DIV BX
MOV [SI+4], AX
MOV [SI+6], DX
HLT
```

Input

Output

Random Access Memory

5000:3000 | update | table | list

```
5000:3000  13 00 03 00 06 00 01 00-00 00 00 00 00 00 00 00
5000:3010  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:3020  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:3030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:3040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:3050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:3060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:3070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Title: Familiarization with 8086 simulator on PC. Study of programs using basic instruction set on the simulator / Trainer Kit.

Program:

```
MOV AX, 5000
MOV DS, AX
MOV SI, 2000H
MOV BL, [1000H]
AND BL, 0FH
MOV [SI], BL
MOV BL, [1000H]
AND BL, 0F0H
ROR BL, 04 H
MOV [SI+01H], BL
HLT
```

Input

original source co...

```
01 MOV AX,5000H
02 MOV DS,AX
03 MOV SI,2000H
04 MOV BL,[1000H]
05 AND BL,0FH
06 MOV [SI],BL
07 MOV BL,[1000H]
08 AND BL,0F0H
09 ROR BL,04
10 MOV [SI+01H],BL
11 HLT
12
13
```

Output

5000:2000 | update | table | list

```
5000:2000  05 02 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2010  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2020  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
5000:2070  [illegible]
```

ment 2: Two-digit BCD numbers is stored in memory location 31000H and 31001H. Pack the BCD and store the packed BCD number in memory locations 32000H.

am:

```
MOV DX, 3000H
MOV DS, DX
MOV SI, 1000H
MOV DI, 2000H
MOV BL, [SI]
ROR BL, 04H
MOV AL, [SI+01H]
OR BL, AL
MOV [DI], BL
HLT
```

ut

original source co...

```
01 MOV AX,3000H
02 MOV DS,AX
03 MOV SI,1000H
04 MOV DI,2000H
05 MOV BL,[SI]
06 ROR BL,04H
07 MOV AL,[SI+01H]
08 OR BL,AL
09 MOV [DI],BL
10 HLT
11
12
```

tput

3000:2000 update table list

```
3000:2000  52 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
3000:2010  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
3000:2020  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
3000:2030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
3000:2040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
3000:2050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
3000:2060  00 00 00 00 00 00 00 00
3000:2070
```

R.......

...ment 3: Write a program in 8086 microprocessors to find out the largest among 8-bit 10 numbers, ...e numbers are stored from memory address 3000: 5000 and store the result (largest number) into ...y address 3000: 6000.

...am:

```
MOV DX, 3000H
MOV DS, DX
MOV SI, 5000H
MOV CL, 0AH
MOV CH, 00H
J2: MOV AL, [SI]
CMP CH, AL
JNC J1
MOV CH, AL
J1: INC SI
DEC CL
JNZ J2
MOV [6000H], CH
HLT
```

...ut

Random Access Memory

3000:5000 | update | table | list

3000:5000	01	05	07	08	0A	0D	06	00	0B	09	00	00	00	00	00	00
3000:5010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:5020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:5030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:5040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:5050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:5060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

...tput

Random Access Memory

3000:6000 | update | table | list

3000:6000	0C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:6010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:6020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:6030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:6040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:6050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3000:6060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

1.

```
MOV A,#14H
MOV B,#43H
ADD A,B
MOV R5,A

MOV A,#32H
MOV B,#62H
ADDC A,B
MOV R1,A
END
```

2.A

```
MOV A,#34H      ; Low byte of 1234H
MOV B,#78H      ; Low byte of 5678H
ADD A,B
MOV 70H,A       ; Store low byte result

MOV A,#12H      ; High byte of 1234H
MOV B,#56H      ; High byte of 5678H
ADDC A,B
MOV 71H,A       ; Store high byte result

END
```

2.B

```
CLR C           ; Clear borrow

MOV A,#78H      ; Low byte of 5678H
MOV B,#34H      ; Low byte of 1234H
SUBB A,B
MOV 72H,A
```

```
MOV A,#56H      ; High byte of 5678H
MOV B,#12H      ; High byte of 1234H
SUBB A,B
MOV 73H,A

END
```

3.A

```
MOV A,#25H
MOV B,#04H

MUL AB

MOV 70H,A     ; Low byte
MOV 71H,B     ; High byte
```

3.B

```
END

MOV A,#25H
MOV B,#04H

DIV AB

MOV 72H,A     ; Quotient
MOV 73H,B     ; Remainder

END
```